



长文本

一、长文本的核心问题与解决方向

1.1 文本长度与显存及计算量之关系

要研究清楚长文本的问题，首先应该搞清楚文本长度在模型中的地位与影响。那么我们便以 Decoder-base 的模型为例来进行分析

1.1.1 模型参数量

Decoder-base 的模型主要包括 3 个部分：embedding, decoder-layer, head。

其中最主要部分是decoder-layer，其由 l 个层组成，每个层又分为两部分：self-attention 和 MLP。

self-attention 的模型参数有 Q 、 K 、 V 的权重矩阵 W_Q 、 W_K 、 W_V 及bias，输出矩阵 W_O 及bias，4个权重矩阵的形状为 $[h, h]$ （ h 表示 hidden_size），4个bias的形状为 $[h]$ 。则 self-attention 的参数量为 $4h^2 + 4h$ 。

MLP 由2个线性层组成，一般地，第一个线性层是先将维度从 h 映射到 $4h$ ，第二个线性层再将维度从 $4h$ 映射到 h 。第一个线性层的权重矩阵 W_1 的形状为 $[h, 4h]$ ，偏置的形状为 $[4h]$ 。第二个线性层权重矩阵 W_2 的形状为 $[4h, h]$ ，偏置形状为 $[h]$ 。则 MLP 的参数量为 $8h^2 + 5h$ 。

self-attention 和 MLP 各有一个layer normalization，包含了2个可训练模型参数：缩放参数 γ 和平移参数 β ，形状都是 $[h]$ 。2个layer normalization的参数量为 $4h$ 。

由此，每个 Decoder 层的参数量为 $12h^2 + 13h$ 。

此外，**embedding** 和 **head** 的参数量相同，与词表相关，为 Vh （如果是 Tied embedding，则二者共用同一个参数）。由于位置编码多样，且参数量小，故忽略此部分。

综上， l 层模型的可训练模型参数量为 $l(12h^2 + 13h) + 2Vh$ 。当 h 较大时，可以忽略一次项，模型参数量近似为 $12lh^2$ 。

1.1.2 计算量估计

如果说参数量是模型的固有属性，那么计算量便是由模型和输入共同决定，下面分析这一过程。

假设输入文本经过分词器后的 input_ids 数据的形状为 $[b, s]$ （ b 表示 batch_size, s 表示 sequence_length），然后经过 embedding 层后 $\times$ 形状变为 $[b, s, h]$ ，由于仅仅是查表操作，计算量为 bsh 。

先分析 Decoder 中 **self-attention** 的计算量，计算公式如下：

$$Q = xW_Q, K = xW_K, V = xW_V$$
$$x_{out} = softmax(\frac{QK^T}{\sqrt{h}}) \cdot V \cdot W_o + x$$

1. 计算 Q, K, V ：矩阵乘法的输入和输出形状为 $[b, s, h] \times [h, h] \rightarrow [b, s, h]$ 。计算量为 $3 * 2bsh^2 = 6bsh^2$ 。

3. 计算在 V 上的加权 $\text{score} \cdot V$ ，矩阵乘法的输入和输出形状为

$[b, n_head, s, s] \times [b, n_head, s, head_dim] \rightarrow [b, n_head, s, head_dim]$ 。计算量为 $2bs^2h$ 。

4. attention后的线性映射，矩阵乘法的输入和输出形状为 $[b, s, h] \times [h, h] \rightarrow [b, s, h]$ 。计算量为 $2bsh^2$ 。

接下来分析MLP块的计算，计算公式如下：

$$x = f_{\text{gelu}}(x_{\text{out}} W_1) W_2 + x_{\text{out}}$$

1. 第一个线性层，矩阵乘法的输入和输出形状为 $[b, s, h] \times [h, 4h] \rightarrow [b, s, 4h]$ 。计算量为 $8bsh^2$ 。

2. 第二个线性层，矩阵乘法的输入和输出形状为 $[b, s, 4h] \times [4h, h] \rightarrow [b, s, h]$ 。计算量为 $8bsh^2$ 。

将上述计算量相加，得到每个Decoder层的计算量大约为 $24bsh^2 + 4bs^2h$ 。

此外，另一个计算量的大头是logits的计算，将隐藏向量映射为词表大小。矩阵乘法的输入和输出形状为 $[b, s, h] \times [h, V] \rightarrow [b, s, V]$ ，计算量为 $2bshV$ 。

因此，对于一个 l 层的模型，输入数据形状为 $[b, s]$ 的情况下，一次前向计算的计算量为 $l * (24bsh^2 + 4bs^2h) + 2bshV$ 。

1.1.3 文本长度与计算量、参数量、显存的关系

忽略低次项，一次输入的tokens数为bs，则计算量与参数量的关系为 $\frac{l * (24bsh^2 + 4bs^2h)}{bs * 12lh^2} = \frac{6h+s}{3h}$ ，在实际中通常 $s < h$ (在长文本情况下，可能会出现 $s > h$ 的情况，但影响不是决定性的)，因此该项可近似认为约等于 2。即在一次前向传递中，对于每个token，每个模型参数，需要进行2次浮点数运算(一次乘法运算和一次加法运算)。考虑到后向传递的计算量是前向传递的2倍。因此一次训练迭代中，对于每个token，每个模型参数，需要进行 $2 * 3 = 6$ 次浮点数运算。

通过以上分析，我们可以得到结论：计算量主要和模型参数和 token 数相关，文本长度并不会显著增加计算量。那么这就引出另一个问题：文本长度与显存的关系。

除了模型参数、梯度、优化器状态外，占用显存的大头就是前向传递过程中计算得到的中间激活值。这里的激活(activations)指的是：前向传递过程中计算得到的，并在后向传递过程中需要用到的所有张量。

先分析 Decoder layer 中 self-attention 的中间激活：

1. 对于 Q, K, V ，需要保存它们共同的输入 x ，这就是中间激活。输入 x 的形状为 $[b, s, h]$ ，元素个数为 bsh ，占用显存大小为 $2 * bsh = 2bsh$ 。

2. 对于 QK^T 矩阵乘法，需要保存中间激活 Q, K ，两个张量的形状都是 $[b, s, h]$ ，占用显存大小合计为 $2 * 2 * bsh = 4bsh$ 。

3. 对于 $\text{softmax}()$ 函数，需要保存函数的输入 QK^T ，占用显存大小为 $2bs^2a$ ，这里的 a 表示注意力头数。

4. 计算完 $\text{softmax}()$ 函数后，会进行dropout操作。需要保存一个mask矩阵，mask矩阵的形状与 QK^T 相同，占用显存大小为 bs^2a 。

5. 计算在 V 上的attention，即 $\text{score} \cdot V$ ，需要保存 score ，大小为 $2bs^2a$ ；以及 V ，大小为 $2bsh$ 。二者占用显存大小合计为 $2bs^2a + 2bsh$ 。

6. 计算输出映射以及一个dropout操作。输入映射需要保存其输入，大小为 $2bsh$ ；dropout需要保存mask矩阵，大小为 bsh 。二者占用显存大小合计为 $3bsh$ 。

因此，将上述中间激活相加得到，self-attention 的中间激活占用显存大小为 $11bsh + 5bs^2a$ 。

接下来分析 Decoder layer 中 MLP 的中间激活：

对于第二项位置编码输入参数，目前暂定为 bs^2a 。

4. 最后有一个dropout操作，需要保存mask矩阵，占用显存大小为 bsh 。

对于MLP块，需要保存的中间激活值为 $19bsh$ 。

另外，self-attention块和MLP块分别对应了一个layer normalization。每个layer norm需要保存其输入，大小为 $2bsh$ 。2个layer norm需要保存的中间激活为 $4bsh$ 。

综上，每个层需要保存的中间激活占用显存大小为 $34bsh + 5bs^2a$ 。对于 l 层transformer模型，还有embedding层、最后的输出层。embedding层不需要中间激活。总的而言，当隐藏维度 h 比较大，层数 l 较深时，这部分的中间激活是很少的，可以忽略。因此，对于 l 层模型，中间激活占用的显存大小可以近似为 $(34bsh + 5bs^2a) * l$ ，这个结果与文本长度关系密切。

下面以GPT3-175B为例，对比下文本长度对模型参数与中间激活的显存大小的影响。假设数据类型为 FP16。

模型名	参数量	层数	隐藏维度	注意力头数
GPT3	175B	96	12288	96

GPT3的模型参数量为175B，占用的显存大小为 $2 \times 175 \times 10^9 bytes = 350GB$ 。GPT3 模型需要占用350GB的显存。

假设 GPT3 输入的 $b = 1$ 。对比不同的文本长度下占用的中间激活：

当 $s = 2048$ 时，中间激活占用显存为 $(34bsh + 5bs^2a) * l = 275,414,777,856bytes \approx 275GB$ ，大约是模型参数显存的0.79倍；

当 $s = 4096$ 时，中间激活占用显存为 $(34bsh + 5bs^2a) * l = 937,376,612,352bytes \approx 937GB$ ，大约是模型参数显存的2.68倍。

可以看到长度仅仅到 4K，显存占用就出现了剧烈增加，同时 GPU 的 onchip memory 就显得更加捉襟见肘(因此也就出现了 FlashAttention 这类算法)。因此如何解决长文本带来的巨量显存开销成为关键及核心问题。

1.2 长文本问题的解决思路

当前，为了实现更长文本的支持，解决思路主要可以分为两个阶段：

- 阶段一：在预训练阶段尽可能支持更长的文本长度

为实现这一阶段目标，通常采用并行化 (parallelism) 方法将显存占用分摊到多个 device，或者改造 attention 结构，避免显存占用与文本长度成二次关系。

- 阶段二：在 SFT 或推理阶段尽可能外推到更大长度

为实现这一阶段目标，通常也是需要在两个方面进行考虑：

- 对位置编码进行外推
- 优化 Attention 机制

本文接下来的部分将尽可能详细深入地进行这些问题的研究。为了便于理解和接受，下文将从易到难，先介绍第二阶段的技术，然后再介绍第一阶段（同时也是考虑到直接使用开源模型者，不需要第一阶段的情况）。

二、长文本与位置编码

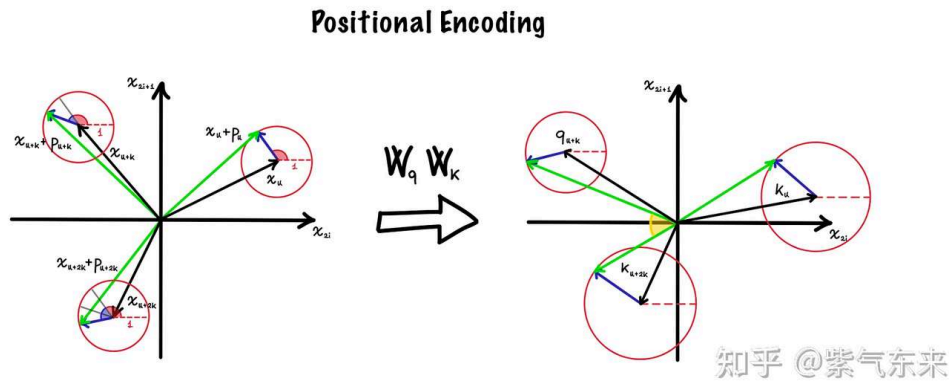
在 Transformer 结构的模型中，Attention模块的值与顺序无关，因此需要加入位置编码以确定不同位置的 token。典型的位置编码方式有两类：

- 绝对位置编码：即将位置信息融入到输入中

2.1 绝对位置编码及其外推

一般来说，绝对位置编码会加到输入中：在输入的第 k 个输入向量 $\mathbf{x}_k$ 中加入位置向量 $\mathbf{p}_k$ 得到 $\mathbf{x}_k + \mathbf{p}_k$ ，其中 $\mathbf{p}_k$ 仅依赖于位置 k 。

如下图所示，以二维向量为例来形象说明，图左中黑色箭头为输入向量 $\mathbf{x}_k$ ，蓝色箭头为位置向量 $\mathbf{p}_k$ （不同方法的长度与角度不同），其相加的结果为绿色箭头。在 Attention 结构中， $\mathbf{q} = (\mathbf{x}_k + \mathbf{p}_k)W_q, \mathbf{k} = (\mathbf{x}_k + \mathbf{p}_k)W_k$ 即相当于同时对输入向量 $\mathbf{x}_k$ 和位置向量 $\mathbf{p}_k$ 进行线性变换，那么 attention 值则是 $\mathbf{q}$ 和 $\mathbf{k}$ 的点积，如下图右中 $\mathbf{q}_{u+k}$ 和 $\mathbf{k}_{u+2k}$ 的注意力即黄色夹角区域。

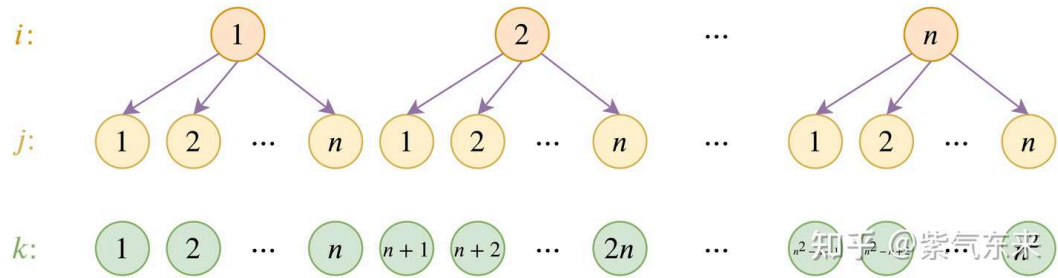


添加图片注释，不超过 140 字（可选）

由于绝对位置编码由两部分组成，且两部分相互独立，因此无法计算相对距离。下面介绍几种典型的绝对位置编码：

2.1.1 训练式编码

这种方式最为简单直接，即把位置当做词表一样，训练一个 $[max_length, hidden_size]$ 位置向量矩阵。这种训练式的绝对位置编码，一般的认为它没有外推性，但是苏剑林大神提出过一个层次分解的拓展方法。



添加图片注释，不超过 140 字（可选）

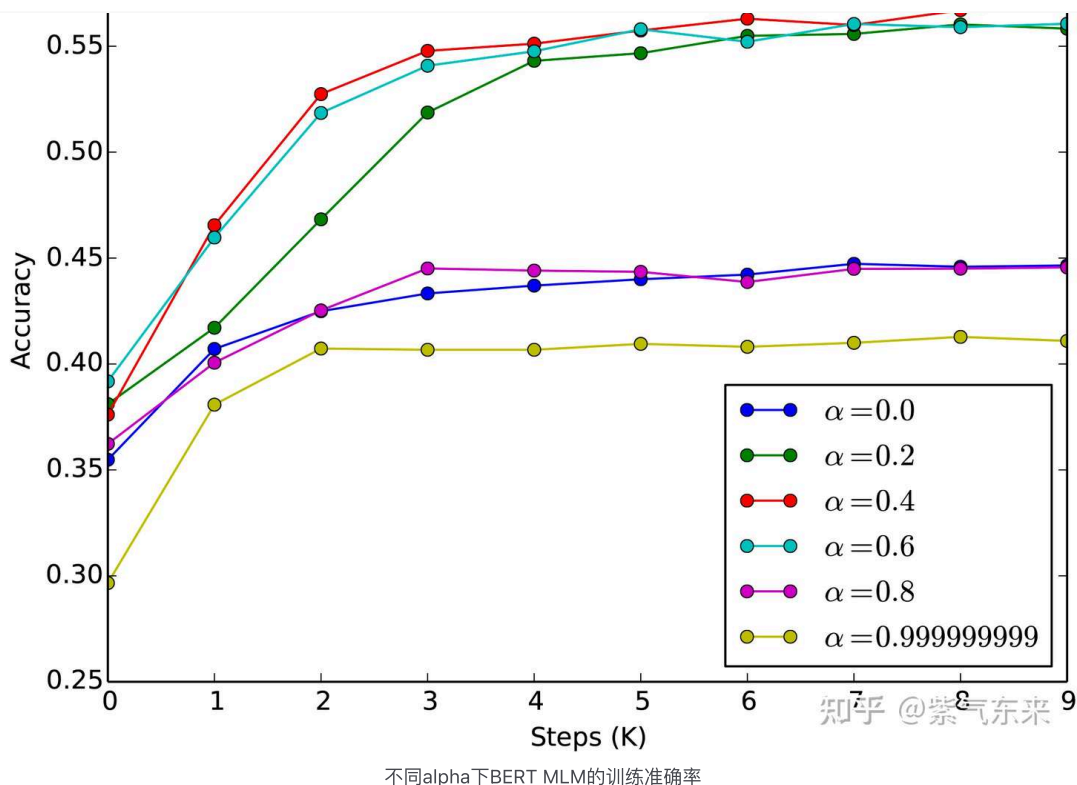
假设已经训练好的绝对位置编码向量为 $\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n$ ，希望能在此基础上构造一套新的编码向量 $\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_m$ ，其中 $m > n$ 。为此，设

$$\mathbf{q}_{(i-1) \times n + j} = \alpha \mathbf{u}_i + (1 - \alpha) \mathbf{u}_j$$

其中超参 $\alpha \in (0, 0.5) \cup (0.5, 1)$ ， $\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_n$ 是该套位置编码的“基底”。为了保障 $\mathbf{q}_1 = \mathbf{p}_1, \mathbf{q}_2 = \mathbf{p}_2, \dots, \mathbf{q}_n = \mathbf{p}_n$ ，这样就能反推出各个 $\mathbf{u}_i$ ：

$$\mathbf{u}_i = \frac{\mathbf{p}_i - \alpha \mathbf{p}_1}{1 - \alpha}, \quad i = 1, 2, \dots, n$$

这样就最大可以表示出 n^2 个位置的编码，并且前 n 个位置编码跟原来模型是相容的。下图反映了经过finetune其准确率在延长的位置编码在MLM任务是行之有效的。



需要说明的是，这种矩阵式的位置编码方式在当前的大模型中已经比较少采用了，仅有 GPT2 等早期模型中采用了这种方式。

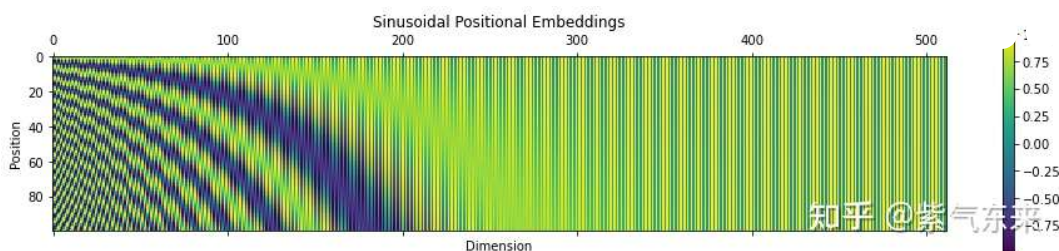
2.1.2 Sinusoidal 位置编码

这种方案也是 Attention Is All You Need 中提出的方法

$$\begin{cases} p_{k,2i} = \sin(k/10000^{2i/d}) \\ p_{k,2i+1} = \cos(k/10000^{2i/d}) \end{cases}$$

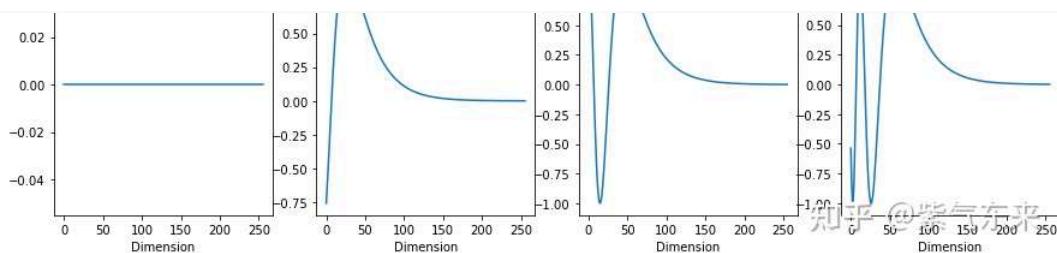
其中 $p_{k,2i}, p_{k,2i+1}$ 分别是位置 k 的编码向量的第 $2i, 2i+1$ 个分量， d 是位置向量的维度。

根据以上定义，我们可以非常简单计算得到Sinusoidal位置编码的值，并绘制图像研究其规律。整体位置编码如下图所示：

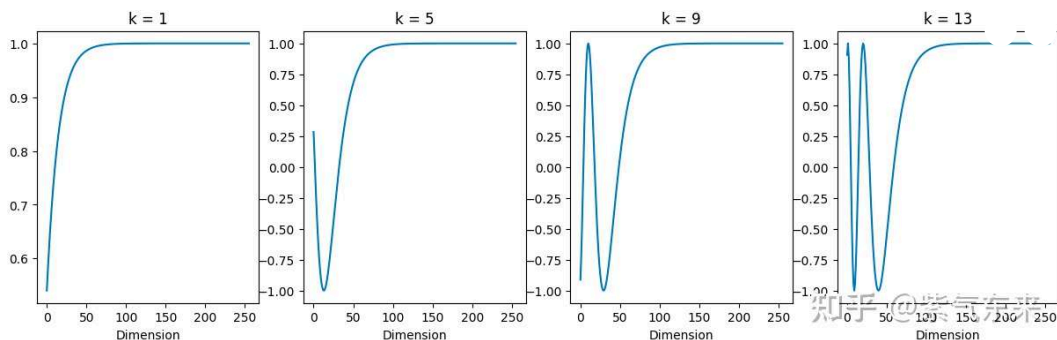


首先研究 Sinusoidal 位置编码与位置之间的关系，绘制不同维度下，函数值与位置的关系。其曲线如下图所示，可以从图中得到几点结论：

- 位置越远，频率越大
- 随着维度增大，函数逐渐收敛到 0（cos 函数收敛到 1）



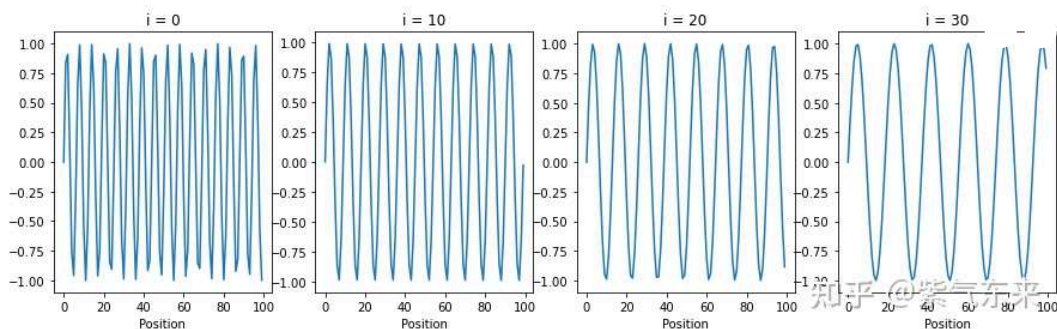
位置编码值在维度偶分量下与位置的关系



位置编码值在维度奇分量与位置的关系

研究 Sinusoidal 位置编码与维度分量之间的关系，绘制不同维度分量 i 下，函数值与位置的的关系。Sinusoidal 位置编码与维度分量的关系如下图所示，可以发现结论如下：

- 每个分量都具有周期性，是正弦或余弦函数
- 越靠后的分量(i 越大)，波长越长，频率越低



添加图片注释，不超过 140 字（可选）

了解了这些基本的特性后，接下来就需要讨论更加深层次的问题：

问题一：为什么用包含各频率的正弦和余弦对？

位置编码存储的是一个包含各频率的正弦和余弦对，这样做有两个好处：

- 可以使得不同位置的编码向量之间有一定的规律性，比如相邻位置之间的差异较小，而距离较远的位置之间的差异较大。这是由正弦和余弦函数的连续性和单调性保证的，即对于任意两个相邻的位置，它们对应的编码向量在每一个维度上都只有微小的变化，而对于任意两个距离较远的位置，它们对应的编码向量在每一个维度上都有较大的差异。
- 可以使得编码向量在任意维度上都能保持唯一性，即不同位置在同一个维度上不会有相同的值。这是由正弦和余弦函数的周期性和相位差保证的，即对于任意两个不同的位置，它们对应的编码向量在每一个维度上都不相等。

问题二：底数对结果的影响是什么？

底数越大，位置向量能表示的序列就越长，这是大底数的好处。但是，底数大，意味着在-1到+1的范围内向量的取值越密集，造成两个位置的向量距离越近，这对后续的Self-Attention模块来说是不利的，因为它需要经历更多的训练次数才能准确地找到每个位置的信息，或者说，才能准确地区分不同的位置。长序列需要长编码。但这样又会

由于

$$\begin{aligned}\sin(\alpha + \beta) &= \sin \alpha \cos \beta + \cos \alpha \sin \beta \\ \cos(\alpha + \beta) &= \cos \alpha \cos \beta - \sin \alpha \sin \beta\end{aligned}$$

这表明位置 $\alpha + \beta$ 的向量可以表示成位置 α 和位置 β 的向量组合，这提供了位置拓展的可能性。

2.1.3 其他的绝对位置编码

如递归式（如 FLOATER）和相乘式（如 PENG Bo: 中文语言模型研究: (1) 乘性位置编码），因使用较少，在此不予赘述。

2.2 相对位置编码及其外推

相对位置并没有完整建模每个输入的位置信息，而是在算 Attention 的时候考虑当前位置与被 Attention 的位置的相对距离，由于自然语言一般更依赖于相对位置，所以相对位置编码通常也有着更好的表现，灵活性也更大。

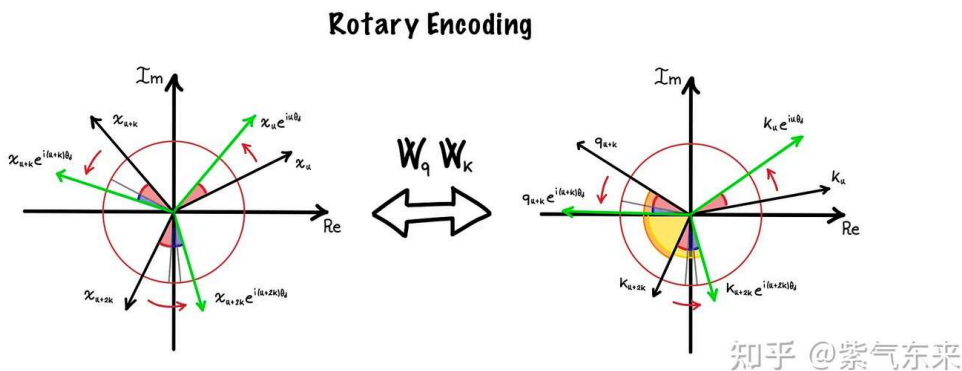
2.2.1 旋转位置编码 RoPE

实际上 RoPE 的诸多思想来源于 Sinusoidal 位置编码，区别在于 Sinusoidal 位置编码采用和 word embedding 相加的形式，RoPE 则采用了矩阵相乘的形式。

在正式介绍之前，我们需要回顾一下经典的欧拉公式

$$e^{i\theta} = \cos \theta + i \sin \theta$$

其矩阵形式为 $\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$ 即旋转矩阵 $R(\theta)$ ，这三种表现形式表达了同样的信息，即将二维向量逆时针旋转角度 θ 。



添加图片注释，不超过 140 字（可选）

接下来我们直接看 RoPE 的表达式，对于位置为 m 的 q 向量，其表达式为

$$f(q, m) = R_m q = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_0 \\ q_1 \end{pmatrix}$$

即逆时针旋转了 $m\theta$ 度，如上图右所示。同理，位置为 n 的 k 向量的表达式为

$$f(k, n) = R_n k = \begin{pmatrix} \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{pmatrix} \begin{pmatrix} k_0 \\ k_1 \end{pmatrix}$$

那么便可以通过点积，计算二者的 attention 值

$$\begin{aligned}
&= q^T \begin{bmatrix} \sin m\theta & \cos m\theta \\ \cos m\theta & \sin m\theta \\ -\sin m\theta & \cos m\theta \end{bmatrix} * \begin{bmatrix} \sin n\theta & \cos n\theta \\ \cos n\theta & -\sin n\theta \\ \sin n\theta & \cos n\theta \end{bmatrix} k \\
&= q^T \begin{bmatrix} \cos n\theta \cos m\theta + \sin n\theta \sin m\theta & \sin m\theta \cos n\theta - \sin n\theta \cos m\theta \\ \sin n\theta \cos m\theta - \sin m\theta \cos n\theta & \cos n\theta \cos m\theta + \sin n\theta \sin m\theta \end{bmatrix} k \\
&= q^T \begin{bmatrix} \cos(n-m)\theta & -\sin(n-m)\theta \\ \sin(n-m)\theta & \cos(n-m)\theta \end{bmatrix} k \\
&= q^T R_{n-m} k
\end{aligned}$$

在极坐标表示下，可以更简单推导出以上结果，即

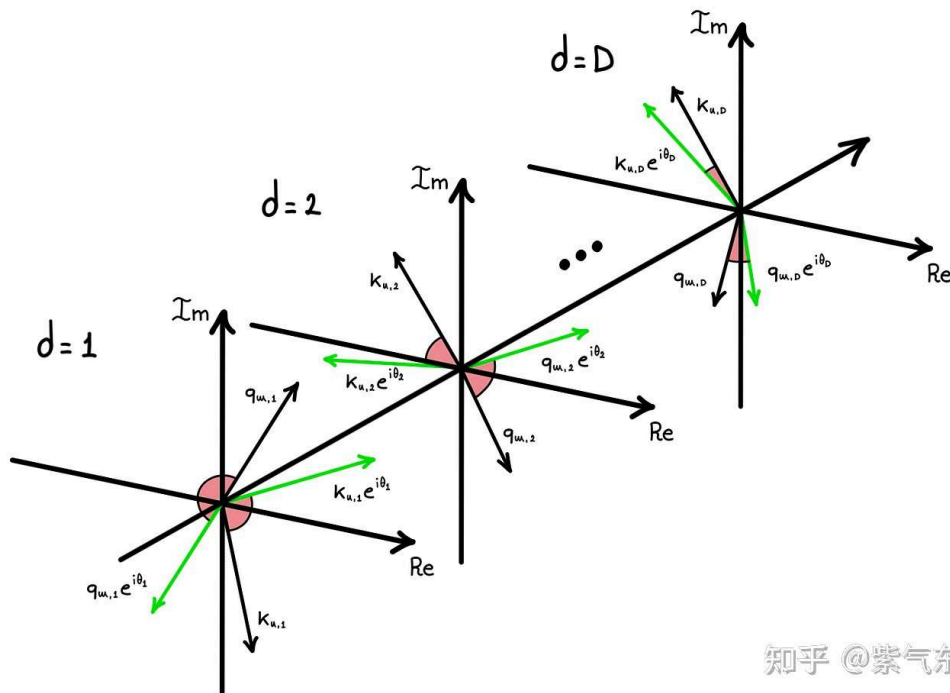
$$\begin{aligned}
q_m \cdot k_n &= f(q, m) \cdot f(k, n) = (R_m q)^T * (R_n k) \\
&= q^T R_m^T * R_n k \\
&= q^T e^{-im\theta_d} * e^{in\theta_d} R_n k \\
&= q^T e^{i(n-m)\theta_d} R_n k \\
&= q^T R_{n-m} k
\end{aligned}$$

即证明了相对位置关系，即旋转前的 attention 值与旋转后的 attention 值的差值仅与相对位置有关。这一点也可以从上图右中看出来，即旋转前 q_{u+k}, k_{u+2k} 的夹角(橙色区域)与旋转后 $q_{u+k}e^{i(u+k)\theta_d}, k_{u+2k}e^{i(u+2k)\theta_d}$ 的夹角(黄色区域)相同，即内积也相同。

这时我们就可以写出位置为 m 的 q 的完整的变换矩阵，即

$$\begin{pmatrix} \cos m\theta_0 & -\sin m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_0 & \cos m\theta_0 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_1 & -\sin m\theta_1 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_1 & \cos m\theta_1 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2-1} & -\sin m\theta_{d/2-1} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2-1} & \cos m\theta_{d/2-1} \end{pmatrix} \begin{pmatrix} q_0 \\ q_1 \\ q_2 \\ q_3 \\ \vdots \\ q_{d-2} \\ q_{d-1} \end{pmatrix}$$

从改变换矩阵也能看出，随着维度增加，旋转角度也在指数级减小，如下图所示。RoPE 的这一功能使模型可以通过从低维度到更高维度，将嵌入中编码的信息类型从低频 (close) 转变为高频 (far)。



旋转角度随维度的变化

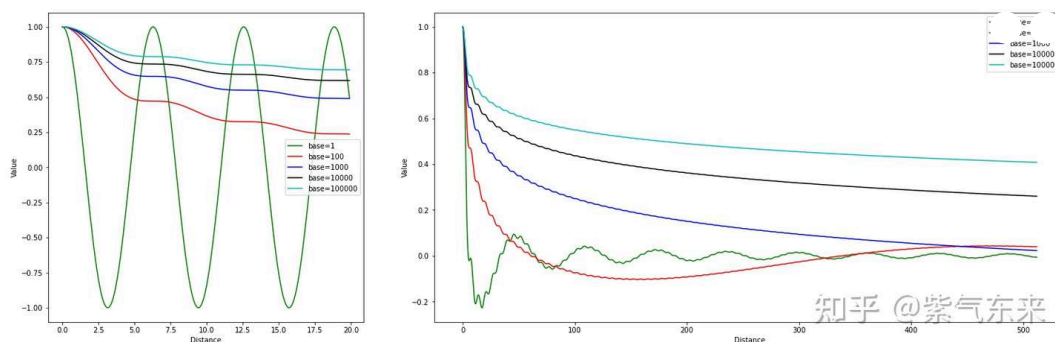
知乎 @紫气东来

$$\begin{aligned}
 \langle \mathbf{p}_m, \mathbf{p}_n \rangle &= \operatorname{Re} \left[e^{-i(m-n)10000^{-1/d/2}} + e^{-i(m-n)10000^{-2/d/2}} + \dots + e^{-i(m-n)10000^{-(d/2)/d/2}} \right] \\
 &= \frac{d}{2} \cdot \operatorname{Re} \left[\sum_{i=0}^{d/2-1} e^{i(m-n)10000^{-i/d/2}} \frac{1}{d/2} \right] \\
 &\approx \frac{d}{2} \cdot \operatorname{Re} \left[\int_0^1 e^{i(m-n) \cdot 10000^{-t}} dt \right]
 \end{aligned}$$

那么问题就变成了积分 $\int_0^1 e^{i(m-n) \cdot 10000^{-t}} dt$ 的渐进估计问题，通过一下函数计算积分值与位置距离的关系，并分析不同 base 值的影响。

下图展示了不同距离尺度上不同 base 值的积分结果，可以得到以下结论：

- 除了 base=1 外，均有明显的远程衰减特性
- base 越小，衰减得越快且幅度也更大
- base 越大，衰减得越慢且幅度也越小



不同距离尺度上的积分结果

2.2.3 RoPE 长度的内插与外推

长度外推性是一个训练和预测的长度不一致的问题。提现有两点：

- 预测的时候用到了没训练过的位置编码（不管绝对还是相对）；
- 预测的时候注意力机制所处理的token数量远超训练时的数量。

一旦我们在模型中有效地整合了相对位置信息，增加 LLM 上下文窗口的最直接方法就是通过位置插值 (position interpolation, PI) 进行微调。

这种方法实现很简单，如果希望将预训练阶段的位置向量范围 $[0, 2048]$ 外推到 $[0, 4096]$ ，只需要将对应位置缩放到原先支持的区间 $[0, 2048]$ 内：计算公式如下， L 为原先支持的长度 (如 2048)， L' 为需要扩展的长度 (如 4096)：

$$f'(x, m) = f(x, \frac{mL}{L'})$$

其过程如下图所示：



添加图片注释，不超过 140 字（可选）

下面分析一下以上操作的本质，经过这种放缩操作后，位置为 m 的维度为 i 的旋转角变为 $\frac{mL}{L'} * base^{-2i/d}$ ，即线性减小了旋转弧度，如下图第一列的上图所示（横轴为位置编码，纵轴为旋转弧度）。通过这种方式插值后，向量旋转速度变慢，周期变大，频率变慢。

除了上述的这种差值方式外，还有以下改进方式可以实现外推：

• NTK-aware (Neural Tangent Kernel)

这种方式把旋转角修改为 $m * (base * \alpha)^{-2i/d}$ ，其中 α 表示 $base$ 的缩放因子，在 codellama 中取值为 100。其修改的方式如下图第二列下图所示（横轴为维度，纵轴为旋转角），在不同维度上修改的程度不同。这种方式保留了高频信息，即

- 高频分量旋转速度降幅低，低频分量旋转速度降幅高；
- 在高频部分进行外推，低频部分进行内插。

这是因为靠前的维度，在训练中见过非常多完整的旋转周期，位置信息得到了充分的训练，所以具有较强的外推能力。靠后的维度，在训练中无法见到完整的旋转周期，或者见到的旋转周期非常少，训练不够充分，外推性能弱，需要进行位置插值。

• NTK-by-parts

该方法是基于 NTK-Aware 的优化，其核心思想是：不改变高频部分，仅缩小低频部分的旋转弧度。即不改变小维度的旋转弧度，仅减小大维度的旋转弧度，这就是 by-parts 的含义。

第 i 个维度的旋转周期为： $\lambda_i = \frac{2\pi}{\theta_i} = 2\pi * base^{2i/d}$

其在训练长度内旋转的周期个数如下： $r_i = \frac{L}{\lambda_i}$

引入超参数 α, β ，表示旋转周期个数的约束条件，定义斜坡函数

$$\gamma(r_i) = \begin{cases} 0, & \text{if } r_i < \alpha \\ 1, & \text{if } r_i > \beta \\ \frac{r_i - \alpha}{\beta - \alpha}, & \text{otherwise} \end{cases}$$

当 $r_i < \alpha$ ，旋转周期数量少，则为低频分组，仅进行线性插值。

当 $\alpha \leq r_i \leq \beta$ ，选择周期介于二者之间，类似于“NTK-aware”插值

当 $r_i > \beta$ ，旋转周期数量足够多，则认为该维度为高频部分，无需改变。

综合以上条件，调整后的旋转角可以表示为 $h(\theta_i) = (1 - \gamma(r_i)) \frac{\theta_i}{s} + \gamma(r_i) \theta_i$

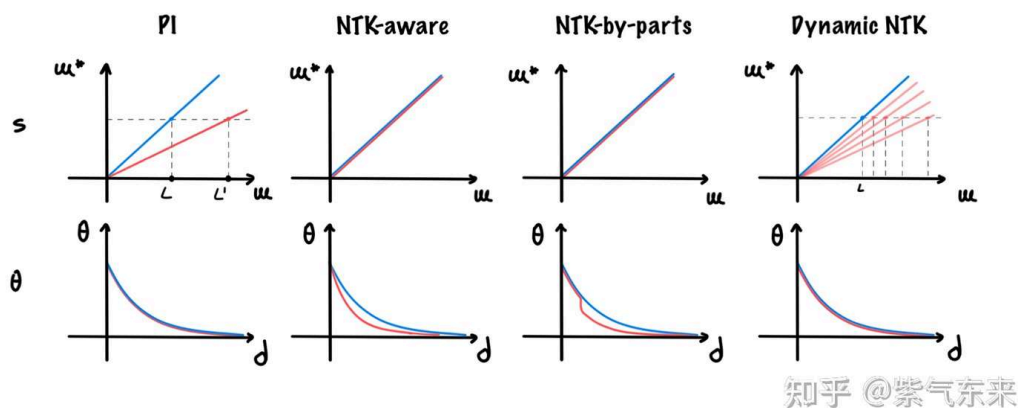
其中 $s = \frac{L'}{L}$ ，超参数的经验取值为 $\alpha = 1, \beta = 32$

l 表示当前的序列长度, L 表示模型训练长度,

当 $l \leq L$ 时, 不调整旋转角

当 $l > L$ 时, 旋转角调整为 $m * (base * \alpha)^{-2i/d}$, 其中 $\alpha = (\frac{l}{L})^{d/(d-2)}$

需要说明的是下图最后一列下图的粗线表示一个范围, 未体现出与长度的动态联动。



PI 及其改进方案

· YaRN

无论是PI还是NTK类方法, 本质都是通过减小旋转弧度, 降低旋转速度, 来达到长度扩展的目的。这将导致位置之间的旋转弧度差距变小, 词向量之间的距离变得比原来更近, 词向量之间的点乘变大, 破坏模型原始的注意力分布。所以经过插值之后, 模型在原来的训练长度内的困惑度均有所提升, 性能受损。

YaRN本质上是NTK-by-parts Interpolation与注意力分布修正策略的结合, 仅缩小低频部分的旋转弧度, 且通过温度系数修正注意力分布。只需将原来的注意力分数除以温度即可。

以LLaMA为例, 当长度从2048扩展至16384时, 长度扩展为原来的8倍, 代入公式, 计算得到。回顾温度系数对注意力分布的影响, 当变大, 注意力分布更加平滑, 方差更小; 当变小, 注意力分布更加尖锐, 区分度变大, 方差变大。意味着缓解注意力分布过于平滑的问题, 让注意力分布方差更大。

需要说明的是, 论文 [Scaling Laws of RoPE-based Extrapolation](#) 中深入研究了 RoPE 位置编码的特性, 其结论就是: RoPE 中 base 的放大和缩小都能获得很好的外推效果(base=10K 效果最差)。原因在于:

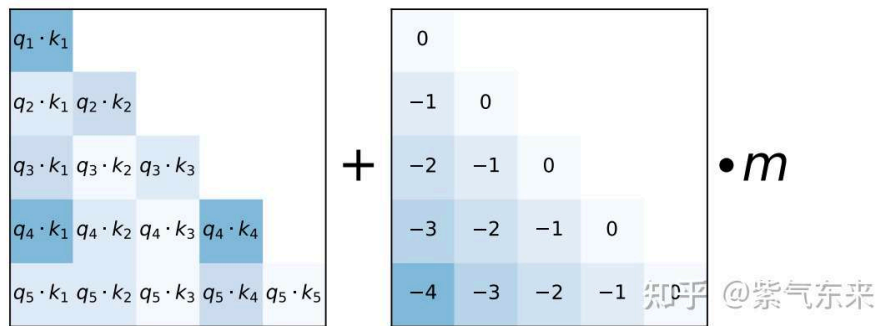
- 当 base 较小时(如 500), RoPE 的三角函数周期变短, 训练时就可以见过完整的 值域;
- 当 base 较大时(如 1000000), RoPE 的三角函数周期变长, 训练时虽然不能见过完整的 值域, 但是外推时仍处于单调区间。

2.2.4 其他形式的编码方式及其外推

在苏神的文章Transformer升级之路: 12、无限外推的ReRoPE? 中指出: RoPE 形式上是一种绝对位置编码, 但实际上给 Attention 带来的是相对位置信息, 即如下的Toeplitz矩阵:

$$\begin{pmatrix} 4 & 1 & 0 & & & & \\ 3 & 2 & 1 & 0 & & & \\ \ddots & 3 & 2 & 1 & 0 & & \\ \ddots & \ddots & 3 & 2 & 1 & 0 & \\ \ddots & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \\ L-2 & \ddots & \ddots & \ddots & \ddots & \ddots & \ddots \\ L-1 & L-2 & \ddots & \ddots & \ddots & 3 & 2 & 1 & 0 \end{pmatrix}$$

这么这种形式的 bias 似乎有种似曾相识的感觉，没错，就是 [ALiBi](#) 编码。严格来说，ALiBi 并不算位置编码，因为它并没有作用在 embedding 上，而是直接作用在了 Attention 上，通过这种构造方式既实现了远程衰减，又实现了位置的相对关系。



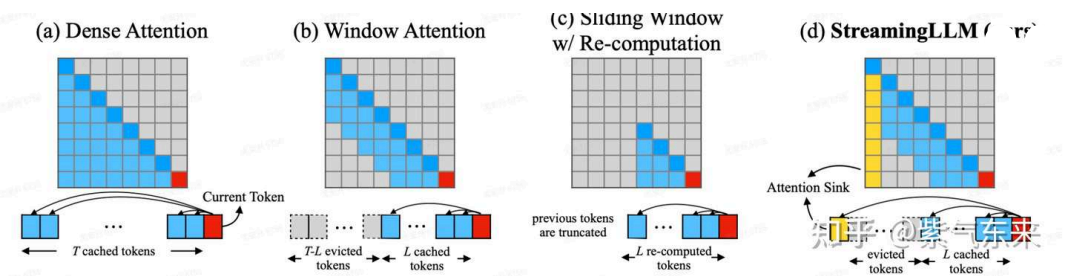
ALiBi 的作用原理

对于外推特性，ALiBi 与前文所述的方法也是不同的，体现在：

- **事后修改**，比如NTK-RoPE、[YaRN](#)、[ReRoPE](#)等，这类方法的特点是直接修改推理模型，无需微调就能达到一定的长度外推效果，但缺点是它们都无法保持模型在训练长度内的恒等性
- **事前修改**，如ALiBi、KERPLE、XPOS以及HWFA等，它们可以不加改动地实现一定的长度外推，但相应的改动需要在训练之前就引入，因此无法不微调地用于现成模型

三、长文本与 Attention 机制

Attention 机制也是制约长文本实现的重要因素，以下是几种典型的 Attention 的方式：



添加图片注释，不超过 140 字（可选）

关于 Attention 机制改进的更多类型和细节，笔者在之前的文章中已经有所讨论，可参考：

紫气东来：NLP（二十）：漫谈 KV Cache 优化方法，深度理解 StreamingLLM
579 赞同 · 18 评论 文章

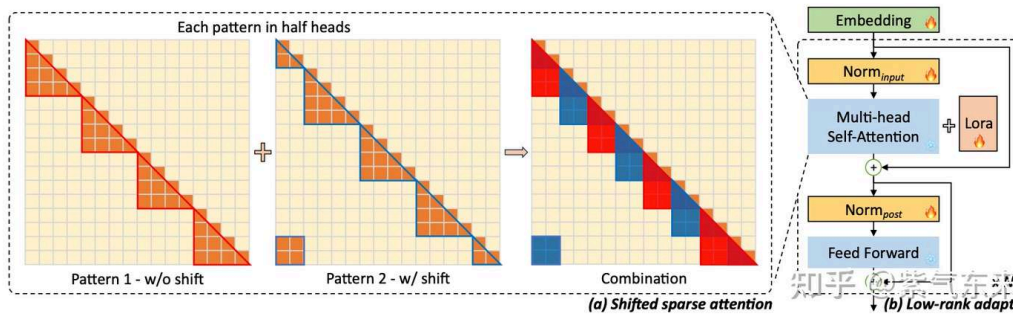


可以双向双向地做训练。

LongLora 在微调期间延长上下文长度，同时使用 Lora 方法保持高性能和低复杂性。其中最关键的是提出了**转移短注意力（S2-Attn）**方案。下面简要介绍这一方案：

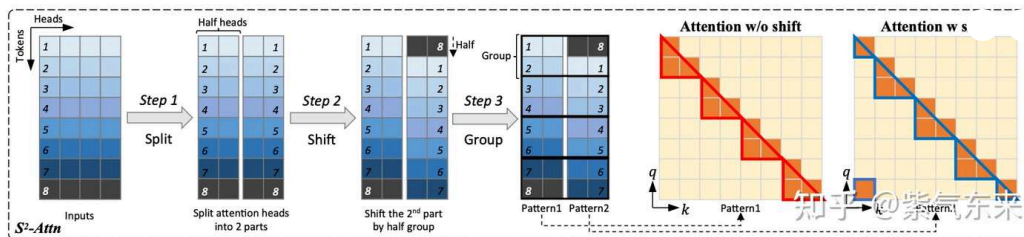
S2-Attn 在微调阶段使用局部注意力而不是全局注意力。即将输入文档分解为几个不同的组，并在每个组中分别应用注意力机制（Pattern 1）。尽管这种方式能够在资源占用不多的情况下拓展长度，由于不同组之间缺乏信息交换，随着上下文长度的增加，会导致混乱增加。

为了解决上述问题，S2-Attn 引入了组大小一半的移位操作，确保相邻组之间顺利的信息交换（Pattern 2）。这种做法有助于模型在文本开头和结尾之间顺利交换信息，从而提高模型稳定性。



LongLora 整体框架

而本文提出的 shift short attention 有一半的 head 会被做 shift，如下图所示，然后每个 group 内作 self-attention，从而使信息可以在不同 group 间传递。这种做法实际上将 Pattern 1 和 Pattern 2 结合起来，而没有引入额外的计算开销，使其非常适合高效处理长序列文本。



shift short attention 原理

此外，LongLoRA相比于Lora还可以微调embedding层和normalization层。尽管这两项内容占的参数量很小（以Llama 2-7B为例，embedding层只占1.94%，normalization层更是不到十万分之四），对结果也起到了重要作用。

四、长文本的预训练方法

上两节主要介绍了如何在位置编码和 attention 机制方面进行文本长度的有效拓展，这两个方面都是“经济适用性”的，即只需要简单微调或者直接外推即可，接下来将是最困难，也是成本最高的部分，即讨论如何在预训练阶段提高文本长度。

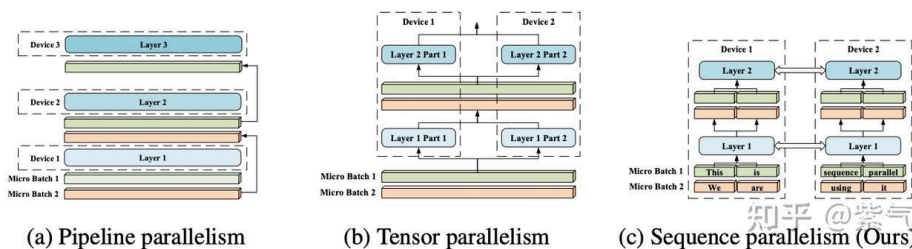
为解决预训练过程中的长文本问题，思路主要有以下几个方面：

- 并行化计算，典型方法如 [sequence parallelism](#)
- 优化 attention 机制，典型方法如 [Transformer-XL](#), [Longformer](#)
- 引入 memory 机制，典型方法如 [Focused Transformer](#), [Memorizing Transformer](#)
- 引入采样机制，典型方法如 [Hierarchical transformers](#), [Dynamic-Pooling Transformer](#)

由于笔者精力有限，下面仅选取其中部分方法加以介绍。

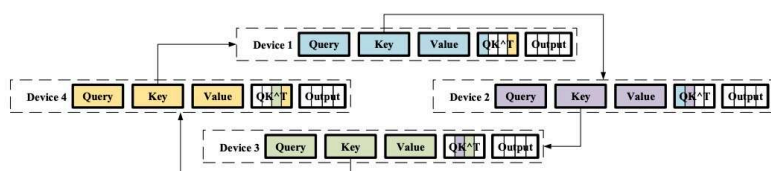
4.1 序列并行(sequence parallel)

简单来说，SP 就是将一段完整的文本拆分到多个设备上计算，设备在适当的时候进行通信和信息交互，如下图(c)所示。

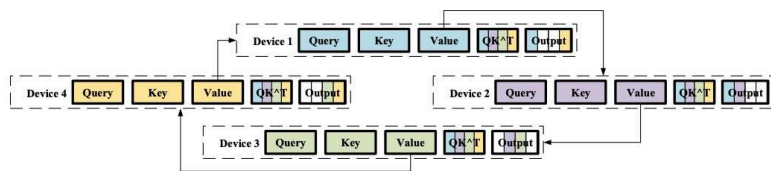


添加图片注释，不超过 140 字（可选）

在实现层面，借鉴了 Ring-Allreduce 的思想，将输入序列分割成多个块，并将每个块输入到其相应的设备中。为了计算注意力输出，将环状通信与自注意力计算相结合，实现了环自注意力（RSA），如下图所示。



(a) Transmitting key embeddings among devices to calculate attention scores



(b) Transmitting value embeddings among devices to calculate the output of attention layers

Figure 2: Ring Self-Attention

添加图片注释，不超过 140 字（可选）

下面我们来深度理解一下这个过程，论文中的符号表示为

- B: batch size
- L: sequence length
- H: hidden size of linear layers
- A: attention head size
- Z: number of attention heads
- N: number of GPUs

对于 $\frac{L}{N}$ ，

切分后的每一个小块 $\frac{L}{N}$ ，在和 $\frac{H}{A}$ 矩阵乘后得到 $\frac{H}{A}$ ，需要在切分维度做补全才可以得到每一个小块的完整结果，在后续进行 $\frac{H}{A}$ 操作。故需要做 ring 操作达到一个 $\frac{H}{A}$ 的操作。

对于 $\frac{H}{A}$ ，

需要完整对 $\frac{H}{A}$ 结果进行 value 查询，需要对 $\frac{H}{A}$ 根据序列并行度进行分块，用对应的块在对应的 value 上查询并求和，

parallelism	Ring-CA	$(D, Z, \frac{L}{N}, A)$	$(D, Z, \frac{L}{N}, A)$	$(D, Z, \frac{L}{N}, L)$	$10AZH + \frac{N}{N}$
	Ring-AV	$(B, Z, \frac{L}{N}, L)$	$(B, Z, \frac{L}{N}, A)$	$(B, Z, \frac{L}{N}, A)$	$+ \frac{BZL^2}{N} + \frac{BLH}{N}$
	Linear	$(B, Z, \frac{L}{N}, A)$	(AZ, H)	$(B, \frac{L}{N}, H)$	知乎 @紫气东来

添加图片注释，不超过 140 字（可选）

MLP 部分的计算就更简单了，如下所示：

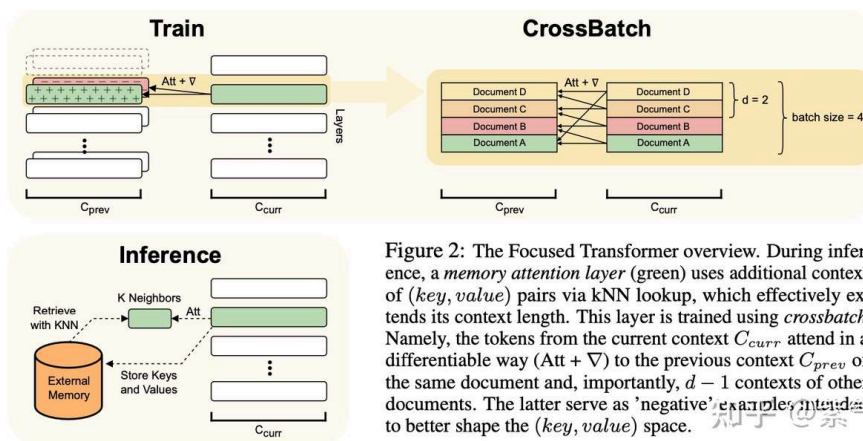
Sequence parallelism	1st linear	$(B, \frac{L}{N}, H)$	$(H, 4H)$	$(B, \frac{L}{N}, 4H)$	$32H^2 + \frac{5BLH}{N}$
	2nd linear	$(B, \frac{L}{N}, 4H)$	$(4H, H)$	$(B, \frac{L}{N}, H)$	

添加图片注释，不超过 140 字（可选）

4.2 LongLLaMA (Focused Transformer)

LongLLaMA 通过引入 Focused Transformer (FOT) 方法，在保持性能的同时，将 LLaMA 的上下文长度扩展到 100k！在长文本的情况下，除了第一节所研究的显存和计算量的问题外，这篇论文还提出了一个分心问题 (Distraction Issue)，即随着文本长度的增加，其中相关的 tokens 对不相关 tokens 的比例会减少，从而导致与不相关 value 相关的 key 和与相关 value 相关的 key 发生重叠，致使模型需要额外区分不同语义的 key。

为此文章提出了 **Focused Transformer** (FOT) 解决方案，其中主要使用了 Memory Attention Layers 以及 CrossBatch 技术，在 Inference 的过程中，绿色的 Memory Attention Layers 使用 kNN 对外部的 Memory 进行查询，从而有效延长了上下文长度，而 Memory Attention Layers 则主要使用 CrossBatch 进行训练。



添加图片注释，不超过 140 字（可选）

具体而言，Memory Attention Layers 中的每个 query 在 Inference 中会关注局部的上下文以及 Memory 中使用 kNN 计算出的最匹配的 key，而整个 Memory 则根据之前处理的 key, value 进行填充。而 CrossBatch 则期望使得 Memory Attention Layers 更加关注长文本之中的“相关 value 的 key”，CrossBatch 的处理借鉴了对比学习的思想，以相关文档之中的 $d-1$ 个上下文作为正样本，以不相关文档之中的 $d-1$ 个上下文作为负样本，通过对比学习的方式使得 Memory Attention Layers 可以更好的分辨相关与无关的 key-value。

与标准的 Transformer 相比，一般的 Transformer 的训练过程中，相关与不相关文档没有被得到有效区分（正负样本分散均匀），当文档数量扩展时，注意力变得越来越分散，而 Focused Transformer 则通过 CrossBatch 的训练目标使得模型有效的关注与区分长文本下的相关与无关的 key-value 的空间结构，从而解决了分心的问题。

五、长文本的效果评估

5.1 PPL

对于 LLM 来说，其效果通常是通过生成连贯且上下文相关的文本的能力来衡量的。为了量化和衡量这一指标，困惑度 (Perplexity, PPL) 便成了最常见的指标。

PPL 是使用平均交叉熵计算的，而平均交叉熵又是使用数据集中的单词数量和根据前面的上下文预测的单词（目标单词）的概率来计算的。前面的上下文通常由目标单词之前的固定长度单词序列表示。其公式如下：

其中H是平均交叉熵，

PPL 作为一种客观的评估指标被广泛用来进行 LLM 的评估。但是其也存在一些问题和不足：

- 模型词汇量可能会不公平地影响PPL：PPL 在很大程度上依赖于模型的词汇量及其概括未见过的单词的能力。如果模型遇到训练数据中不存在的单词或短语，即使生成的文本有意义，其 PPL 分数也较高。
- 缺乏主观性考虑：PPL 是一种客观指标，不考虑主观因素，例如风格、创造力或特定环境下的适当性。
- 上下文理解：PPL 主要关注于根据前面的上下文预测下一个单词。然而，它可能无法捕捉模型对更广泛背景的整体理解。
- 语言歧义和创造力：PPL 并不能体现模型处理语言歧义或生成创造性和新颖输出的能力。
- 领域特异性：PPL 对训练数据的领域和分布很敏感。在特定领域训练的模型可能会在其领域内实现较低的复杂性，但可能需要帮助在其训练环境之外生成文本。
- 过度拟合和泛化：PPL可能会受到过度拟合的影响，其中模型在训练数据上表现得非常好，但很难泛化到看不见的或现实世界的例子。

实际上，StreamingLLM 就很好地证明了 PPL 的局限性，因为尽管 StreamingLLM 的 PPL 值较低，但是由于其损失了大量中间信息，因此无法在“大海捞针”等测试方法中有较好的表现。

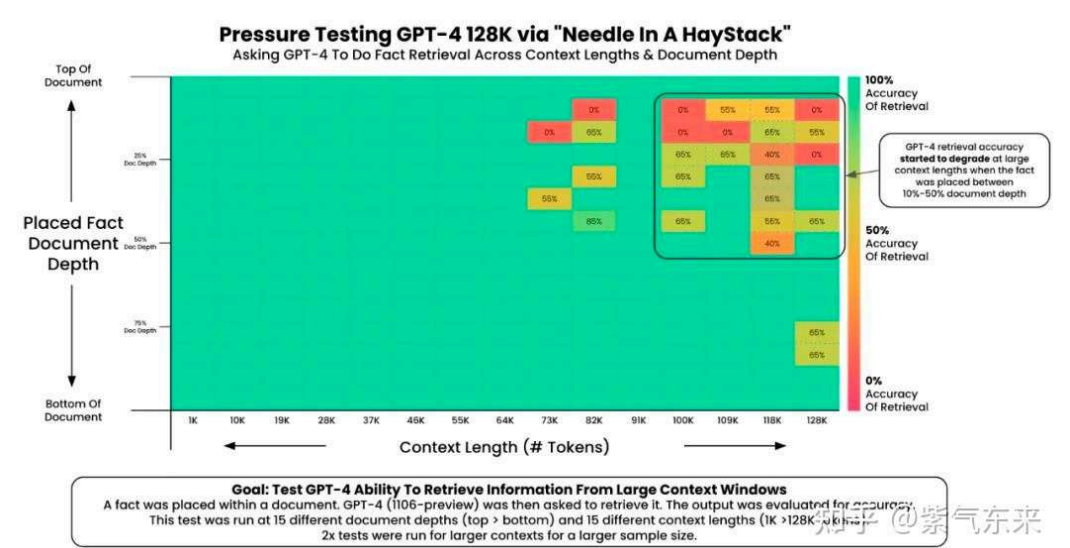
5.2 “大海捞针”

“大海捞针”由 Greg Kamradt 提出的大模型长文本性能测试方法，其做法是在文本语料中藏入一个与文本语料不相关的句子，然后看大模型能不能通过自然语言提问的方式（Prompt）把这句话准确地提取出来。Greg Kamradt 的“大海捞针”实验简述：

“大海”：Paul Graham 的文章合集作为语料
“针”：“The best thing to do in San Francisco is eat a sandwich and sit in Dolores Park on a sunny day.”
提问：“What is the most fun thing to do in San Francisco based on my context? Don't give information outside the document”
期待模型输出的正确答案：
The best thing to do in San Francisco is eat a sandwich and sit in Dolores Park on a sunny day.

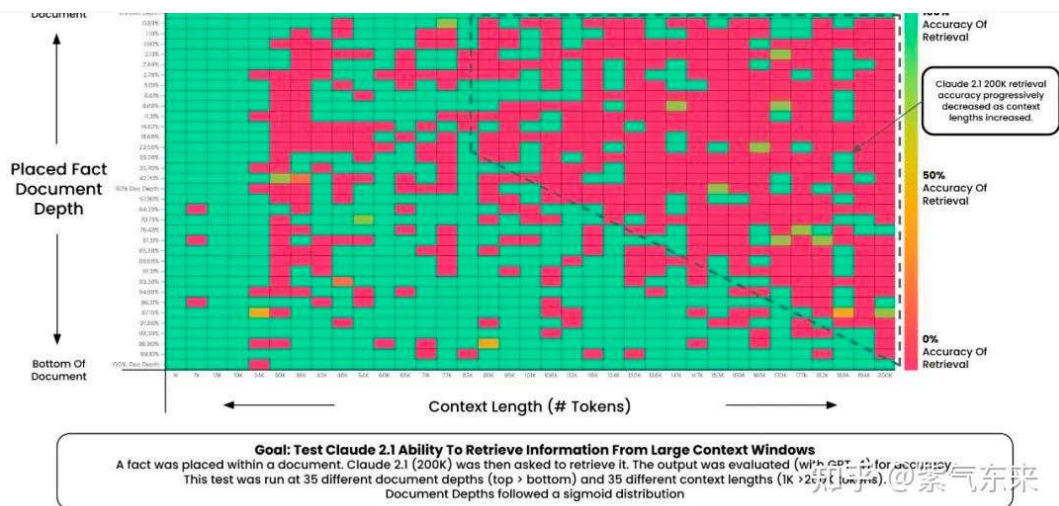
Greg Kamradt 公布了他对 GPT-4 Turbo（128K）和 Claude 2.1 的测试结果：

- GPT-4 Turbo（128K）在语料长度超过 72K 且句子（“针”）藏在文本头部的时候，准确率不佳。



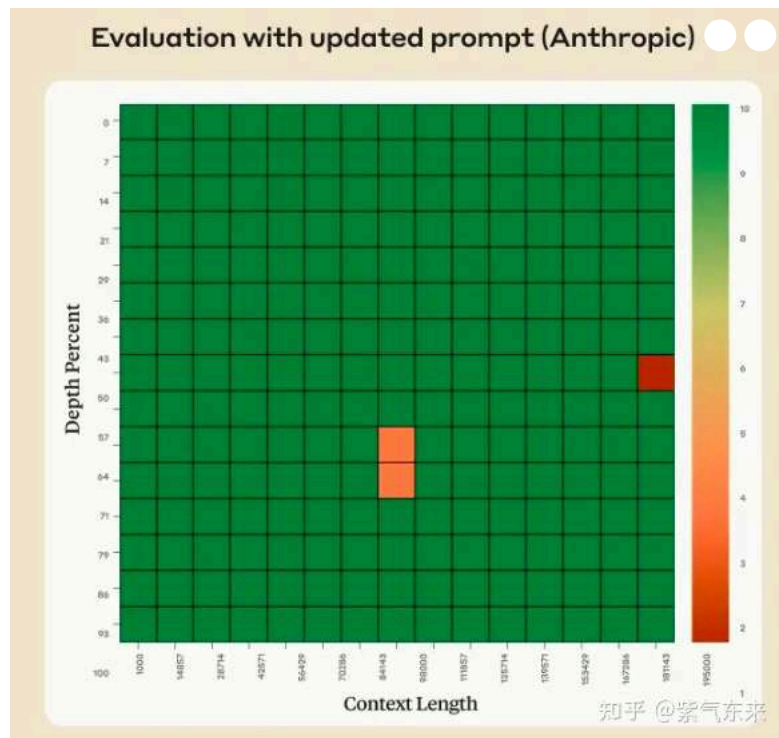
添加图片注释，不超过 140 字（可选）

- Claude 2.1似乎在语料长度超过20K之后就开始准确率不佳，而且句子（“针”）藏在语料靠前的位置时，准确率



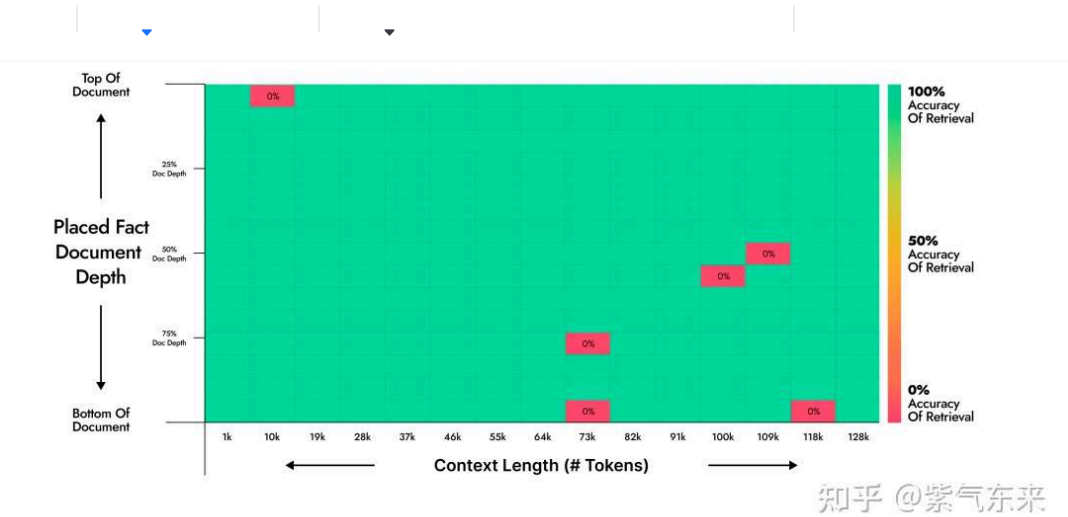
添加图片注释，不超过 140 字（可选）

- 进一步的，Anthropic发现可以通过简单的prompt提示就可以提高模型不愿意回答不相关内容的效果，即让模型回答问题之前，加上一句“**Here is the most relevant sentence in the context:**”即可大幅提升模型回答效果，改进模型不愿意回答不相关内容的水平。



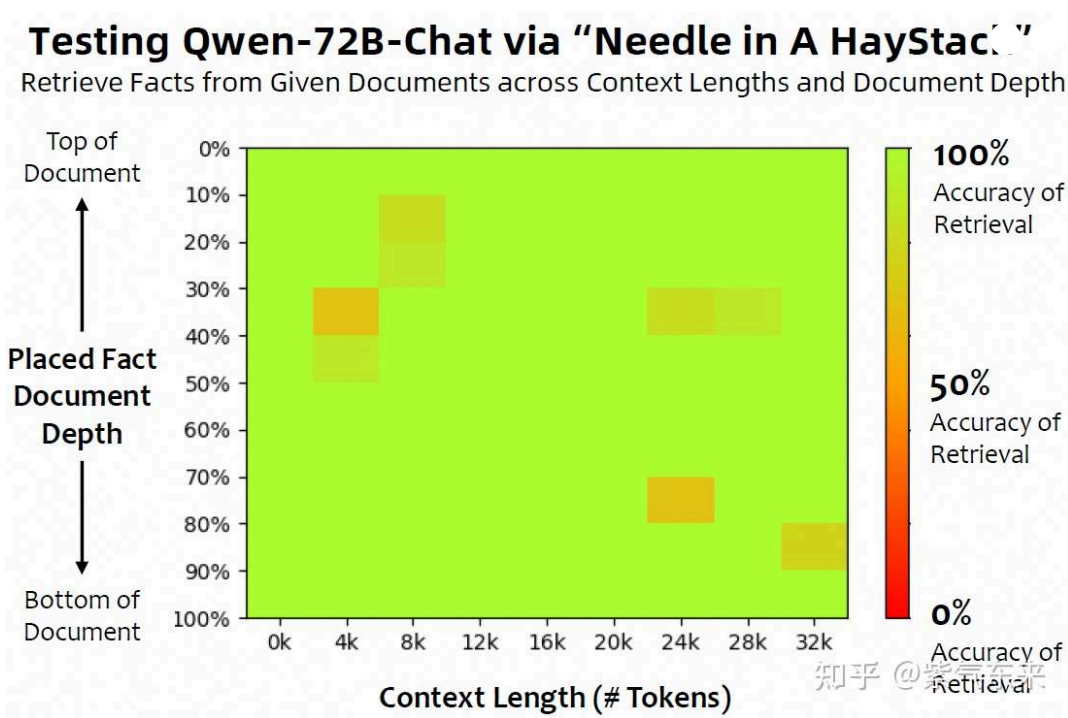
添加图片注释，不超过 140 字（可选）

- 此外国内的 Moonshot AI 的长文本模型 Kimi Chat 也在“大海捞针”实验中发挥了令人惊艳的表现，原始报道见[这里](#)。



添加图片注释，不超过 140 字（可选）

• Qwen-72B-Chat 也进行了“大海捞针”实验，测试模型在不同长度的输入下，是否能检索到文章不同位置的信息，结果同样表现良好，如下：



添加图片注释，不超过 140 字（可选）

发布设置

添加封面

+ 添加文章封面

图片上传格式支持 JPEG、JPG、PNG

投稿至问题 ?

未选择

创作声明 ?

无声明

* 文章话题 ?

+ 添加话题

根据自媒体投稿规范，发布新闻类的内容需要明确标注信息来源，请确认当前内容是否已正确添加

+ 添加来源